



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发

项目名称：CareerPilot——基于多智能体协作
的 AI 求职成长助手系统

方向一：Agentic AI 原生开发

学号：2025302940

姓名：张炜霄

专业：计算机技术

指导教师：戚欣

2026 年 6 月 22 号

目录

- 一、选题背景与设计思想3
- 二、Specs 规格文档.....4
 - 2.1 Product Spec4
 - 2.2 Architecture Spec.....5
 - 2.3 API Spec.....7
- 三、系统架构与设计.....10
- 四、关键实现与代码展示12
 - 4.1 Agent 核心代码及 FastAPI 接口代码12
 - 4.2 Tools 核心代码12
 - 4.3 AI IDE 的使用.....13
 - 4.4 配置文件14
- 五、测试与评估.....16
- 六、系统升级与扩展.....22
- 七、课程总结24

一、选题背景与设计思想

随着大语言模型技术的发展，AI 已经从传统的问答工具逐渐演变为能够执行复杂任务的 Agent 系统。在实际求职过程中，求职者往往需要经历简历准备、岗位筛选、能力评估、技能提升以及面试准备等多个环节，而这些环节通常相互独立，需要依赖不同的平台和工具完成。对于缺乏经验的学生而言，最大的困难并不是学习某一项技能，而是不知道当前能力与目标岗位之间究竟存在什么差距，也不知道应该按照什么顺序进行学习和准备。

目前市面上虽然已经存在一些简历优化工具、岗位分析工具以及模拟面试平台，但这些产品大多只覆盖求职流程中的单一环节。例如简历优化工具只能帮助修改简历，面试平台只能进行面试训练，而无法根据个人能力情况制定成长路线。用户需要频繁切换平台，数据无法共享，整个求职过程缺乏连续性和个性化指导。

基于上述问题，本项目设计并实现了 CareerPilot——一个基于多智能体协作架构的 AI 求职成长助手系统。系统以用户的简历和目标岗位为输入，通过多个 Agent 的协同工作，实现简历解析、岗位分析、能力差距评估、学习路线规划以及模拟面试等功能，形成完整的求职成长闭环。相比传统单功能工具，CareerPilot 更强调 Agent 之间的信息传递和任务协同，使系统能够持续围绕用户的职业目标提供智能化支持。

在技术路线方面，项目采用 SDD (Specification Driven Development) 规格驱动开发思想，在正式编码之前先完成需求规格、架构规格以及接口规格设计。后端采用 FastAPI 构建 REST API 服务，前端使用 Streamlit 快速搭建交互界面，大模型能力由 DeepSeek API 提供支持。系统内部采用多 Agent 架构，每个 Agent 负责独立任务，并通过统一的数据结构完成协作。同时引入 SQLite 实现长期记忆存储，并通过 Docker 支持项目部署与复现。

二、Specs 规格文档

本项目遵循 SDD 方法论进行开发。在项目开始阶段，首先编写 Product Spec，明确系统目标用户为求职学生和初级开发者，核心需求包括简历解析、岗位分析、能力评估、学习规划以及面试训练。系统的最终目标是帮助用户形成完整的求职成长路径，而不仅仅提供单一功能。

以下内容为节选。

2.1 Product Spec

产品名称：

CareerPilot —— AI 求职成长助手

产品定位：

CareerPilot 是一个面向求职学生和初级开发者的 AI Agent 系统，通过分析用户简历与目标岗位需求，自动发现能力差距，生成学习路线，并提供模拟面试与答案评估服务，帮助用户完成从岗位匹配到能力提升的全过程。

目标用户：

①应届毕业生②实习求职者③转行进入软件开发行业的学习者④希望提升岗位竞争力的开发者

核心问题：

①无法准确理解目标岗位的能力要求；②不清楚自身能力与岗位之间存在什么差距；③不知道应该优先学习哪些技能；④缺乏真实面试训练和反馈机制；⑤求职流程割裂，需要使用多个工具完成不同任务。

产品目标：

构建一个能够贯穿求职全过程的 AI 智能助手，实现：

1. 自动解析简历；
2. 自动分析岗位需求；
3. 自动评估岗位匹配度；
4. 自动规划学习路线；
5. 自动生成模拟面试题；
6. 自动评价用户回答并给出反馈。

功能需求：

Resume Agent：解析用户简历并提取结构化信息

Job Agent：分析岗位描述并提取岗位要求

Gap Analysis Agent：对比能力与岗位要求生成差距报告

Learning Agent：根据差距报告生成学习路线

Interview Agent：生成面试题并评估用户回答

Memory Repository：保存用户学习记录与面试历史

非功能需求：

支持网页交互；

支持 PDF 和 DOCX 简历解析；

支持 DeepSeek 大模型调用；

支持 Docker 部署；

支持单元测试；

支持环境变量配置 API Key；

保证模块间低耦合设计。

2.2 Architecture Spec

总体架构

系统采用前后端分离的多智能体架构。

用户通过 Streamlit 前端上传简历并输入目标岗位信息，前端通过 HTTP 请求调用 FastAPI 后端服务。后端负责组织多个 Agent 的协同工作，并通过 DeepSeek API 获得大模型推理能力。系统同时使用 SQLite 实现长期记忆存储。

Agent 职责划分

Resume Agent

负责解析用户简历内容，将非结构化文本转换为结构化数据。

输入：

```
{
  "text": "resume content"
}
```

输出：

```
{
  "skills": [],
  "projects": [],
  "education": [],
  "experience": []
}
```

Job Agent

负责分析岗位描述。

输入：

```
{
  "description": "job description"
}
```

输出：

```
{
  "required_skills": [],
  "preferred_skills": [],
  "experience_level": ""
}
```

Gap Analysis Agent

负责比较用户能力与岗位要求。

输出：

```
{
  "matched_skills": [],
  "missing_skills": [],
  "score": 85
}
```

```
}
```

Learning Agent

负责生成学习路线。

输出：

```
{
  "learning_sequence": [],
  "estimated_duration_weeks": 12,
  "weekly_roadmap": {},
  "recommended_projects": []
}
```

Interview Agent

负责生成面试题和面试评价。

输出：

```
{
  "technical_questions": [],
  "behavioral_questions": []
}
```

以及：

```
{
  "score": 90,
  "feedback": "..."
}
```

LangGraph 状态设计

```
class CareerState(TypedDict):
    resume_text: str
    job_description: str
    resume_info: dict
    job_info: dict
    gap_report: dict
    learning_plan: dict
    interview_questions: list
    memory: dict
```

workflow 设计

START → Resume Agent → Job Agent → Gap Analysis Agent → Learning Agent → Interview Agent → Save Memory → END

Tools 设计

Resume Reader Tool

Input: resume file

Output: resume text

Libraries:

- * pdfplumber
- * python-docx

Skill Extractor Tool

Input: text

Output: skill list

Method: LLM Extraction

Memory Tool

Input: user state

Output: saved state

Backend: SQLite

Evaluation Tool

Input: agent output

Output: score

2.3 API Spec

1. 简历解析接口

Endpoint: POST /analyze-resume

请求:

```
{
  "text": "resume text"
}
```

响应:

```
{
  "success": true,
  "data": {
    "skills": [],
    "projects": []
  }
}
```

2. 岗位分析接口

Endpoint: POST /analyze-job

请求:

```
{
  "description": "job description"
}
```

响应:

```
{
  "success": true,
```

```
"data":{
  "required_skills":[],
  "preferred_skills":[]
}
```

3. 能力差距分析接口

Endpoint: POST /gap-analysis

请求:

```
{
  "resume_info":{},
  "job_info":{}
}
```

响应:

```
{
  "success":true,
  "data":{
    "matched_skills":[],
    "missing_skills":[],
    "score":85
  }
}
```

4. 学习路线生成接口

Endpoint: POST /learning-plan

请求:

```
{
  "gap_report":{}
}
```

响应:

```
{
  "success":true,
  "data":{
    "learning_sequence":[],
    "weekly_roadmap":{},
    "recommended_projects":[]
  }
}
```

5. 面试题生成接口

Endpoint: POST /interview

请求:

```
{
  "job_info":{}
}
```

响应：

```
{
  "success":true,
  "data":{
    "technical_questions":[],
    "behavioral_questions":[]
  }
}
```

6. 面试评分接口

Endpoint: POST /evaluate-answer

请求：

```
{
  "question":"...",
  "answer":"..."
}
```

响应：

```
{
  "success":true,
  "data":{
    "score":87,
    "feedback":"..."
  }
}
```

接口统一规范

所有接口均采用统一响应格式：

```
{
  "success":true,
  "data":{},
  "message":""
}
```

其中 success 表示请求状态，data 表示业务数据，message 表示错误信息或提示信息。

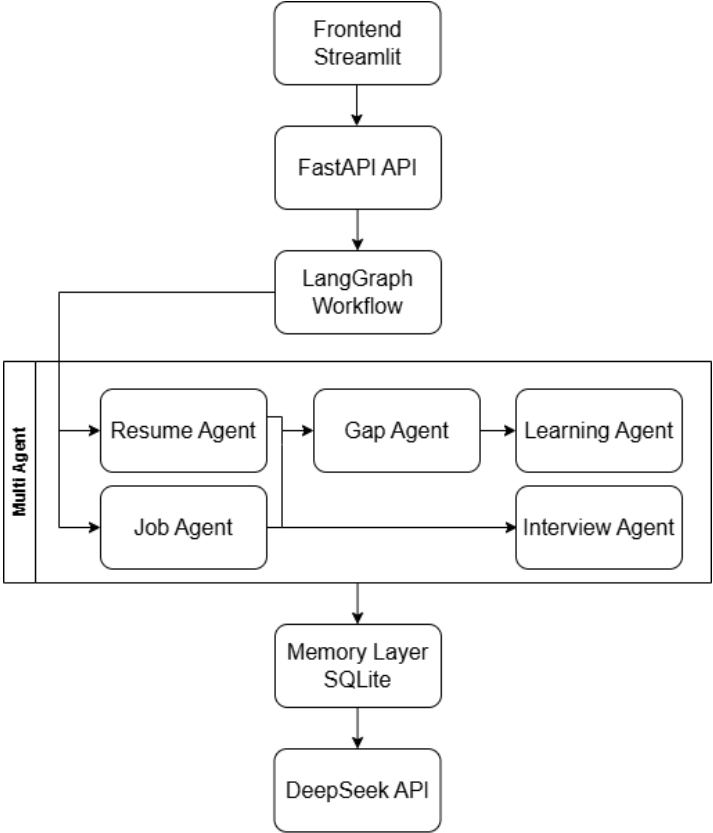
三、系统架构与设计

CareerPilot 采用典型的多智能体协同架构。

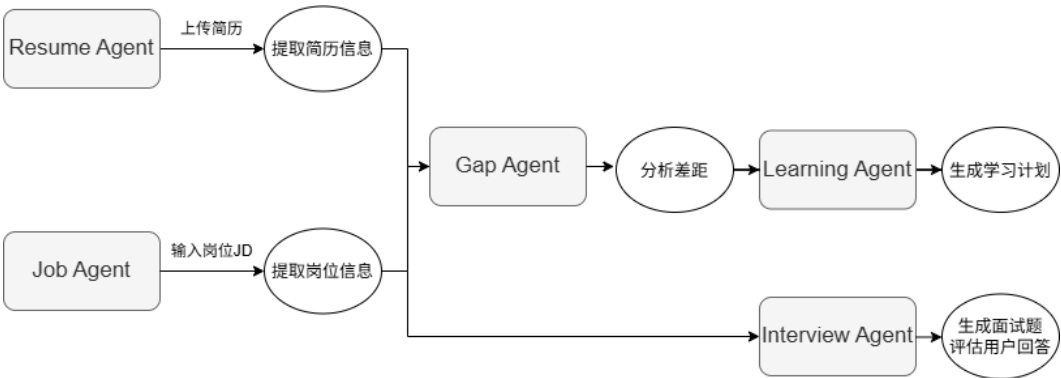
用户首先上传简历文件，系统通过 Resume Agent 将非结构化简历内容转换为结构化信息，包括技能、项目经历、教育背景和工作经验。随后用户输入目标岗位 JD，Job Agent 自动提取岗位要求、技能需求以及经验等级。

当两部分信息准备完成后，Gap Analysis Agent 将用户能力与岗位要求进行对比，识别用户已具备技能与缺失技能，并计算岗位匹配度评分。分析结果将进一步传递给 Learning Agent，由其生成个性化学习路线和项目实践建议。最后，Interview Agent 根据岗位要求自动生成面试题，并对用户提交的答案进行评分和反馈。

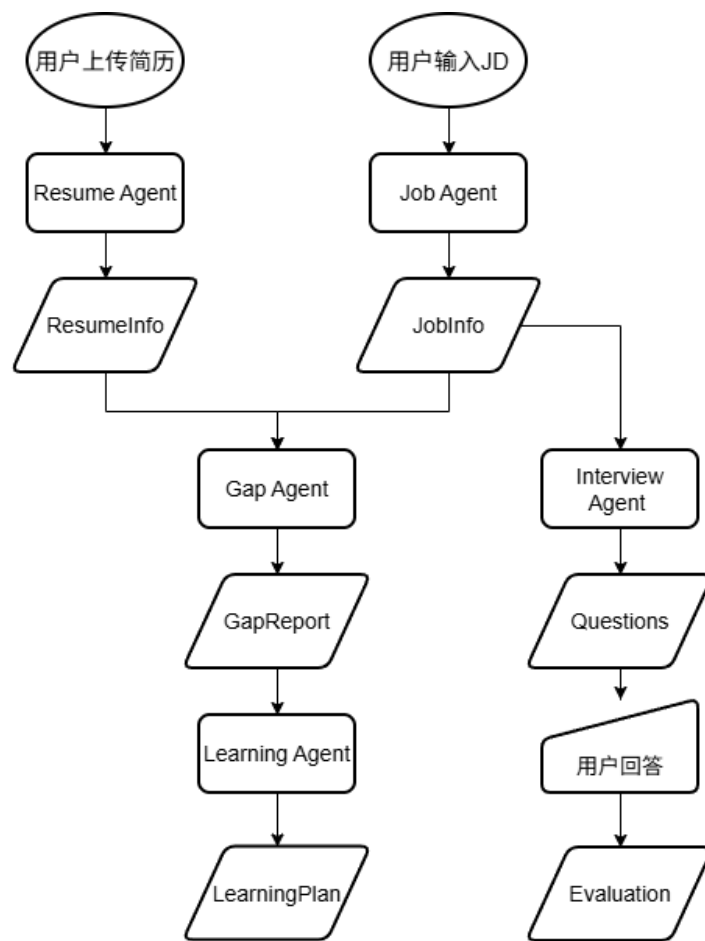
整个系统形成了从“能力评估”到“能力提升”再到“面试验证”的闭环流程。



系统总体架构图-自顶向下



Agent 协作流程图



数据流流程图

从数据流角度来看，系统始终围绕用户职业目标展开。用户上传简历后，Resume Agent 将非结构化文本转换为结构化 ResumeInfo；用户输入目标岗位 JD 后，Job Agent 提取岗位要求形成 JobInfo。两部分数据随后进入 Gap Analysis Agent，用于计算岗位匹配度并生成能力差距报告。GapReport 作为 Learning Agent 的输入，进一步生成学习路线和项目实践建议。最后，Interview Agent 根据岗位要求生成模拟面试题，并结合用户回答输出评分与反馈结果。

整个过程中，各 Agent 之间通过标准化 JSON 数据进行通信，实现了从能力评估到能力提升再到能力验证的闭环流程。

四、关键实现与代码展示

项目的核心实现基于 Agent 模式设计。每个 Agent 负责独立任务，并拥有专属 Prompt 模板和输出结构。系统并非简单调用大模型进行问答，而是通过结构化输出实现任务拆解与结果传递。

4.1 Agent 核心代码及 FastAPI 接口代码

以 Gap Agent 为例，系统会接收 Resume Agent 和 Job Agent 的分析结果，并构建统一 Prompt，引导大模型输出匹配技能、缺失技能以及综合评分。由于各 Agent 的输入输出格式保持一致，因此可以方便地进行串联和扩展。关于前端和 API，前端提交 ResumeInfo 和 JobInfo，FastAPI 调用 GapAnalysis Agent，返回匹配度分析结果。

```
logger = logging.getLogger(__name__)

class GapInfo(BaseModel):
    """Structured gap analysis between resume and job requirements."""
    matched_skills: List[str] = Field(default_factory=list, description="List of skills from the resume that match job requirements")
    missing_skills: List[str] = Field(default_factory=list, description="List of skills required by the job but missing from the resume")
    score: int = Field(default=0, ge=0, le=100, description="Match score from 0 to 100 based on the skillset match")
```

```
prompt = f"""
你是资深技术招聘专家。
将提供的简历信息与职位要求进行比对。
识别出简历中具备的必备技能和优先技能（matched_skills），以及简历中缺失的技能（missing_skills）
并给出一个0到100分的匹配度评分，代表候选人与职位技能要求的契合程度。
只返回JSON。
格式：
{{
    "matched_skills": [],
    "missing_skills": [],
    "score": 0
}}
候选人：
{resume_info}
岗位要求：
{job_info}
"""

response = self.llm_client.invoke(prompt)
result = extract_json(response.content)
return result
```

Gap Agent 核心代码及 prompt 截图

```
@app.post("/gap-analysis", response_model=StandardResponse)
def gap_analysis(req: GapRequest):
    try:
        data = gap_agent.process(req.resume_info, req.job_info)
        return StandardResponse(success=True, data=data, message="Gap analysis completed successfully.")
    except Exception as e:
        return StandardResponse(success=False, data={}, message=str(e))
```

FastAPI 接口代码：Gap Agent 部分

4.2 Tools 核心代码

在工具层面，项目实现了 ResumeReaderTool，用于解析 PDF 与 DOCX 格式简历；SkillExtractorTool 用于提取技能关键词；MemoryRepository 使用 SQLite 实现长期记忆存储；Interview Agent 则负责面试题生成与答案评估。

```

@staticmethod
def _read_pdf(path: Path) -> str:
    text = []
    try:
        with pdfplumber.open(path) as pdf:
            for page in pdf.pages:
                extracted = page.extract_text()
                if extracted:
                    text.append(extracted)
            return "\n".join(text)
    except Exception as e:
        logger.error(f"Error in _read_pdf: {e}")
        raise

@staticmethod
def _read_docx(path: Path) -> str:
    try:
        doc = docx.Document(path)
        return "\n".join([paragraph.text for paragraph in doc.paragraphs])
    except Exception as e:
        logger.error(f"Error in _read_docx: {e}")
        raise

```

ResumeReaderTool 核心代码

```

def extract_skills(self, text: str) -> List[str]:
    """
    Extract a list of skills from arbitrary text.

    Args:
        text: Source text to extract skills from.

    Returns:
        List of skills.
    """
    if not text or not text.strip():
        raise ValueError("Input text cannot be empty")

    try:
        logger.info("Extracting skills using SkillExtractorTool")
        structured_llm = self.llm_client.with_structured_output(SkillList)

        prompt = (
            "Extract all professional and technical skills mentioned in the following text. "
            "Return them as a list of strings.\n\n"
            f"Text:\n{text}"
        )

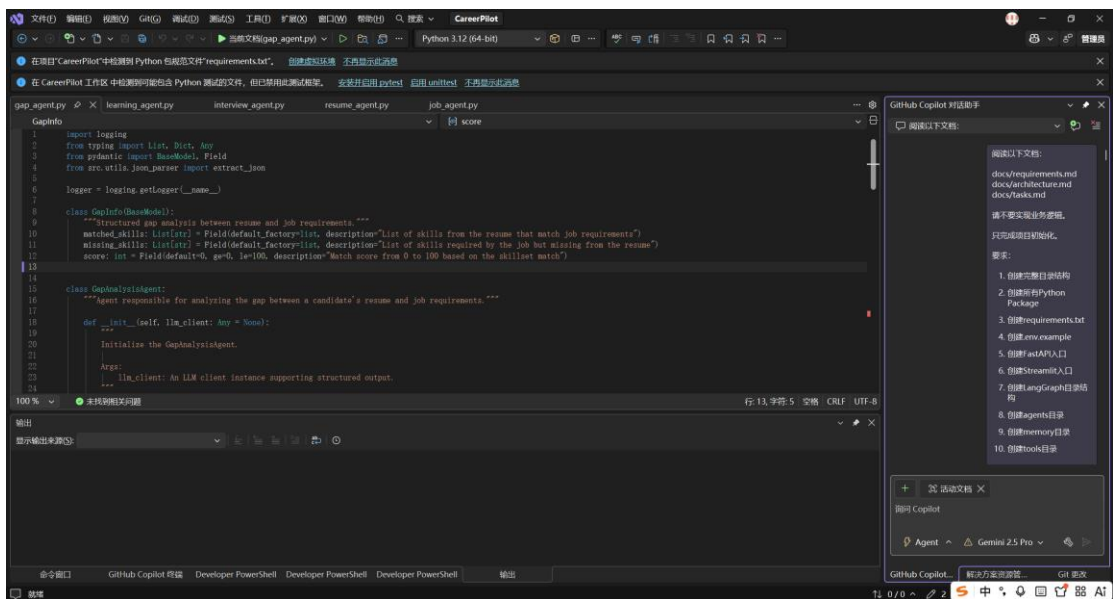
        result: SkillList = structured_llm.invoke(prompt)
        return result.skills
    except Exception as e:
        logger.error(f"Error extracting skills: {e}")
        raise

```

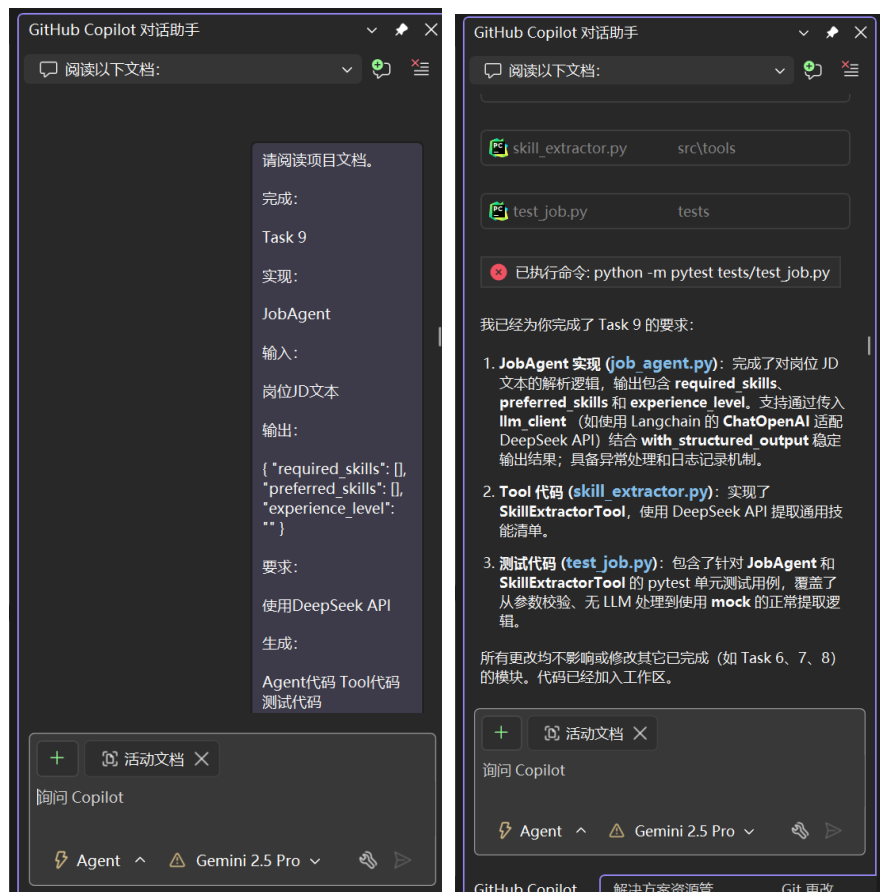
SkillExtractorTool 核心代码

4.3 AI IDE 的使用

开发过程中充分利用 GitHub Copilot 和 AI IDE 完成代码生成、单元测试编写以及模块重构工作。整个项目遵循 AI Native Development 思路，通过规格驱动与 AI 辅助开发相结合的方式快速完成系统实现。



AI IDE 界面截图



AI 代码生成对话内容截图

4.4 配置文件

系统配置采用环境变量方式管理 API Key, 避免敏感信息直接写入代码。项目根目录提供 .env 文件, 用户只需填写 DeepSeek API Key 即可运行系统。

具体内容如下：

DEEPSEEK_API_KEY=

USER_AGENT=CareerPilot/1.0

项目采用 requirements.txt 统一管理依赖环境，其中 FastAPI 用于构建后端服务，Streamlit 用于快速搭建前端界面，LangGraph 和 LangChain 用于 Agent workflow 编排，pdfplumber 与 python-docx 用于简历文件解析，pytest 用于自动化测试。

requirements.txt 包含如下内容：fastapi、uvicorn、streamlit、langgraph、langchain、pydantic、python-dotenv、python-multipart、pdfplumber、python-docx、pytest、langchain-openai、openai

为了提高项目的可复现性和部署效率，本项目还采用 Docker 进行容器化封装。传统 Python 项目在不同计算机环境中部署时，往往会面临 Python 版本不一致、依赖库缺失以及环境配置复杂等问题，而 Docker 能够将应用程序及其运行环境统一封装到镜像中，从而保证开发环境与生产环境的一致性。

项目根目录提供了 Dockerfile 和 docker-compose.yml 配置文件。其中 Dockerfile 负责构建运行镜像，首先基于 Python 运行环境创建容器，然后安装 requirements.txt 中定义的全部依赖库，最后复制项目源代码并启动应用服务。其核心配置如下：

五、测试与评估

项目完成后，对 CareerPilot 的各项核心功能进行了完整测试，以验证系统是否能够实现预期的求职辅助流程。由于本项目属于 AI Agent 系统，其评估不仅包括传统的软件功能测试，还需要关注 Agent 的推理能力、结果质量以及多模块协同效果。因此测试过程主要围绕功能正确性、Agent 行为表现以及系统整体流程三个层面展开。

在功能测试阶段，首先验证了系统各模块的可执行性。项目采用 pytest 编写单元测试，对简历解析、岗位分析、差距分析、学习规划、面试模块以及记忆模块进行了独立测试。测试过程中曾出现多个问题，例如测试框架缺失 pytest-mock 依赖、SQLite 内存数据库导致数据表初始化失败等情况。通过补充依赖、修正数据库初始化逻辑以及修改测试代码中的 Mock 实现后，最终所有测试用例均顺利通过。测试结果表明，各核心模块能够正常完成输入接收、数据处理以及结果返回，系统整体逻辑不存在阻塞性错误。

除了自动化测试之外，还进行了大量人工场景测试。以简历解析模块为例，分别使用包含项目经历、技能描述和教育背景的真实简历进行测试。系统能够正确识别常见开发技能，例如 Python、Java、Spring Boot、MySQL、Docker 等内容，并将非结构化简历文本转换为结构化 JSON 数据。在岗位分析模块中，选取后端开发工程师、Java 开发工程师以及 AI 应用开发工程师等多个真实招聘岗位作为输入，系统能够较准确地提取岗位所要求的技术栈和经验要求。当输入内容过于简单时，例如仅输入“后端开发”四个字，系统也能够给出合理解释，而不会产生程序错误。

能力差距分析模块是整个系统的核心环节，因此进行了重点测试。测试时分别构造高匹配度、中匹配度和低匹配度三类案例。例如，当简历中已经具备 Python、FastAPI、Docker 等技能，而目标岗位要求 Python、FastAPI、Docker 和 Kubernetes 时，系统能够识别前三项为已匹配技能，并将 Kubernetes 标记为缺失技能，同时生成较高的匹配评分。当岗位要求与用户技能完全不相关时，系统能够明显降低匹配度评分并输出更多待提升技能。从实际测试结果来看，匹配技能与缺失技能的识别结果基本符合人工判断，说明 Agent 已具备一定的能力评估能力。

学习规划模块主要评估生成结果的合理性和可执行性。在初始版本中，学习计划固定为四周周期，但实际测试发现不同岗位的技能缺口差异较大，固定周期会导致部分计划不够合理。因此后续修改 Prompt 逻辑，引入根据缺失技能数量动态调整学习周期的机制。例如缺失技能较少时生成 4~6 周计划，而缺失技能较多时则自动扩展至 12 周甚至更长时间。测试结果显示，新的方案能够生成更符合实际学习节奏的路线图，同时给出阶段性学习任务和项目实践建议，提高了学习计划的可执行性。

面试模块测试分为题目生成和答案评估两个部分。在题目生成阶段，系统能够根据不同岗位自动生成对应技术问题。例如针对后端开发岗位生成数据库设计、缓存机制以及微服务架构相关问题，而针对 AI 应用开发岗位则更多关注大模型调用、Prompt Engineering 以及 Agent 系统设计。随后增加了面试评分功能，允许用户输入回答内容并获得评分反馈。评分结果统一采用百分制输出，并附带改进建议。从多次测试结果来看，系统能够根据回答内容的完整性、技术深度以及表达质量给出相对合理的评价。

由于本项目基于 DeepSeek 大模型实现，因此还进行了 Agent 行为测试。测试过程中发现，早期版本采用 LangChain Structured Output 时会出现 API 返回 400 错误的问题，导致 Agent 无法正常工作。经过排查后发现 DeepSeek 当前并不完全支持相关 response_format 配置，因此改为使用 Prompt 约束 JSON 输出并通过程序解析结果。修改后系统稳定性显著提升，所有 Agent 均能够正常完成推理任务。这一过程也说明，在实际 AI 工程开发中，大模型能力本身并不是唯一因素，模型接口兼容性和工程实现同样会直接

影响系统效果。

整体而言，CareerPilot 已经能够完成“简历上传—岗位分析—差距评估—学习规划—模拟面试”的完整闭环测试。用户上传简历后，系统能够自动生成结构化能力画像；输入目标岗位后，可以获得匹配度分析和技能缺口报告；随后生成针对性的学习路线，并最终通过模拟面试验证学习效果。整个流程中各 Agent 能够通过统一数据结构进行协作，未出现数据丢失或模块间调用失败的问题。

本项目虽然规模较小，但已经验证了多 Agent 协作在求职场景中的可行性。测试结果表明，系统具备稳定的功能实现能力，能够完成从岗位匹配分析到职业成长规划的完整业务链路。同时，在实际开发过程中经历的依赖冲突、数据库初始化错误、模型接口兼容性问题以及前后端状态管理问题，也进一步体现了 AI Agent 系统开发与传统软件开发相比所面临的工程挑战。

以下为项目运行示例：

CareerPilot - AI求职成长助手

欢迎使用基于多智能体的求职成长辅助系统。请按照下方标签页顺序进行操作。

1. 上传简历2. 输入岗位3. 差距报告4. 学习路线5. 面试模拟

第一步：上传简历解析

支持上传 PDF 或 DOCX 格式的简历

求职简历.pdf123.jpg

+

开始解析简历

简历解析成功!

```
{  "技能": [    0: "C",    1: "Python",    2: "Keras",    3: "TensorFlow",    4: "YOLOv5",    4: "YOLOv5",    5: "MySQL",    6: "AE",    7: "Photoshop",    8: "Premiere Pro"  ],  "项目经历": [    0: "乳腺癌筛选药物的优化建模 (2022.11 - 2023.01): 小组长, 负责数据预处理、机器学习模型搭建和训练。模型优化, 实现潜在乳腺癌治疗药物的预测",    1: "计算机设计大赛: 人工智能赛道 (2021.12 - 2022.05): 机器人调试, 项目主题为基于优化模型的OpenPose智能陪伴机器人, 负责机器人调试和运行",    2: "口罩识别 (2023.11 - 2024.02): 小组长, 负责YOLOv5模型搭建和训练以及PPT制作",    3: "校园二手交易平台 (2022.09 - 2023.11): 后端开发、小组长, 以MySQL数据库为基础搭建程序框架, 开发主要功能, 并负责PPT制作"  ],  "教育经历": [    0: "武汉理工大学 (211, 双一流) 2020.09 - 2024.06, 数据科学与大数据技术 本科, 计算机与人工智能学院, GPA: 3.678, 专业排名: 30%, 辅修法学, CET4: 635, CET6: 546, 全国大学生英语竞赛三等奖"  ],  "实习经历": []  }
```

Resume Agent：简历解析功能测试

第二步：输入目标岗位 (JD)

请将职位描述 (JD文本) 粘贴到下方

岗位名称: Python后端开发工程师

岗位职责:
1. 负责后端系统开发与维护
2. 设计RESTful API
3. 与前端协作完成项目开发

任职要求:

开始分析岗位

岗位分析成功!

```
{  "必备技能": [    0: "Python"    1: "FastAPI或Django"    2: "MySQL"  ],  "加分技能": [    0: "LangChain"    1: "LLM应用开发"  ],  "经验要求": "1-3年开发经验, 本科以上学历"}
```

Job Agent: 目标岗位需求分析功能测试

第三步：能力匹配度报告

生成匹配度与差距报告

报告生成完成!

能力综合匹配度评分

25 / 100

已具备的优势技能

- Python
- MySQL

待提升的缺失技能

- FastAPI或Django
- Docker
- Git
- LangChain
- LLM应用开发

Gap Agent: 差距分析功能测试

CareerPilot - AI求职成长助手

欢迎使用基于多智能体的求职成长辅助系统。请按照下方标签页顺序进行操作。

1. 上传简历 2. 输入岗位 3. 差距报告 4. 学习路线 5. 面试模拟

第四步：定制学习路线

生成个性化学习路线

学习路线生成成功!

推荐学习顺序

Git → FastAPI或Django → Docker → LangChain → LLM应用开发

双周/月度排期

Week1

- 学习Git基础：安装、配置、常用命令（init, add, commit, push, pull, branch, merge）
- 在GitHub上创建个人仓库，练习提交代码和分支管理

- 完成一个简单的Python脚本（如爬虫或数据处理）并上传至GitHub

Week2

- 学习Git进阶：解决冲突、rebase、stash、tag、PR流程
- 阅读开源项目（如FastAPI官方示例）的Git历史，理解协作流程
- 练习在团队模拟场景中使用Git进行多人协作

Week3

- 学习FastAPI基础：路由、请求/响应模型、依赖注入、Pydantic模型
- 搭建一个简单的RESTful API（如图书管理系统），实现CRUD操作
- 使用MySQL作为数据库，通过SQLAlchemy或ORM连接FastAPI

Week4

- 学习FastAPI进阶：中间件、CORS、文件上传、错误处理、异步处理
- 学习Django基础（可选对比）：MVT模式、ORM、Admin后台
- 选择一个框架（推荐FastAPI）完成一个带用户认证的API项目

Week5

- 学习Docker基础：镜像、容器、Dockerfile、docker-compose

- 将week4的FastAPI项目容器化，编写Dockerfile并运行
- 学习使用docker-compose编排FastAPI + MySQL + Redis服务

Week6

- 学习Docker进阶：多阶段构建、卷挂载、网络配置、镜像优化
- 学习Docker Hub和私有仓库的使用，推送/拉取镜像
- 练习使用Docker部署一个完整的Web应用（含前端静态文件）

Week7

- 学习LangChain基础：Chain、Prompt、Memory、Agent概念
- 使用LangChain调用OpenAI API，实现简单的问答链
- 学习文档加载器（PDF、网页、数据库）和文本分割器

Week8

- 学习LangChain进阶：向量存储（Chroma/FAISS）、检索增强生成（RAG）
- 实现一个基于本地文档的问答系统（如公司知识库）
- 学习使用LangChain的Tool和Agent构建自动化工作流

Week9

- 学习LLM应用开发基础： Prompt Engineering、 温度/采样参数调优
- 学习使用LangChain构建多轮对话系统（带记忆）
- 学习流式输出、回调函数、异步调用等高级特性

Week10

- 学习LLM应用开发进阶： 微调概念、 Embedding、 向量数据库集成
- 学习使用LangChain实现复杂应用： 如SQL查询助手、 邮件生成器
- 学习评估LLM输出质量的方法（如BLEU、 ROUGE、 人工评估）

Week11

- 综合项目： 构建一个智能客服系统（FastAPI + Docker + LangChain + LLM）
- 实现功能： 用户提问 → 检索知识库 → 生成回答 → 记录日志
- 使用Docker部署整个系统，并编写 docker-compose.yml

Week12

- 项目优化： 添加用户认证、 API限流、 错误处理、 性能监控
- 编写项目文档（README、 API文档、 部署指南）
- 将项目上传至GitHub，并准备面试展示（含架构图、 演示视频）

💡 练手项目推荐

- 🚀 ['name': '智能知识库问答系统', 'description': '基于FastAPI + LangChain + Docker构建的RAG系统，支持上传PDF/网页文档，通过LLM回答用户问题。包含用户认证、对话历史、流式输出。', 'skills_covered': ['FastAPI', 'Docker', 'Git', 'LangChain', 'LLM应用开发']]
- 🚀 ['name': '个人博客API + 自动化部署', 'description': '使用FastAPI/Django开发博客后端，集成MySQL数据库，通过Docker容器化部署，使用GitHub Actions实现CI/CD。', 'skills_covered': ['FastAPI或Django', 'Docker', 'Git']]
- 🚀 ['name': 'LLM驱动的数据分析助手', 'description': '用户输入自然语言查询，系统自动生成SQL并查询数据库，返回结果并用LLM生成解释。使用LangChain Agent实现。', 'skills_covered': ['LangChain', 'LLM应用开发', 'FastAPI']]

Learning Agent： 学习路线规划+项目推荐功能测试

CareerPilot - AI求职成长助手 🚀

欢迎使用基于多智能体的求职成长辅助系统。请按照下方标签页顺序进行操作。

1. 上传简历 2. 输入岗位 3. 差距报告 4. 学习路线 5. 面试模拟

🎙️ 第五步：AI 面试模拟

根据 JD 生成专属面试题

面试题目准备就绪！

📖 技术深度问答

- Q1: 请解释Python中装饰器的工作原理，并给出一个实际应用场景。
- Q2: 在FastAPI中，如何实现依赖注入？请举例说明。
- Q3: MySQL中，如何优化一个慢查询？请列出至少三种方法。
- Q4: Dockerfile中，多阶段构建有什么好处？请给出一个简单的示例。
- Q5: Git中，如何处理合并冲突？请描述你的解决步骤。
- Q6: 请解释LangChain中的Chain和Agent的区别，并说明各自的使用场景。
- Q7: 在LLM应用开发中，如何处理上下文窗口限制？请提出至少两种策略。

💡 综合行为考察

- Q1: 请描述一个你在开发中遇到的技术难题，以及你是如何解决它的。
- Q2: 在一个团队项目中，如果同事的代码质量不符合要求，你会如何处理？
- Q3: 请举例说明你如何在1-3年的开发经验中快速学习一项新技术。
- Q4: 当项目截止日期临近，但任务尚未完成时，你如何优先安排工作？

Interview Agent(a)： 面试题生成功能测试

Q4: Dockerfile中，多阶段构建有什么好处？请给出一个简单的示例。

Q4: 当项目截止日期临近，但任务尚未完成时，你如何优先安排工作？

Q5: Git中，如何处理合并冲突？请描述你的解决步骤。

Q6: 请解释LangChain中的Chain和Agent的区别，并说明各自的使用场景。

Q7: 在LLM应用开发中，如何处理上下文窗口限制？请提出至少两种策略。

选择一道面试题

请解释Python中装饰器的工作原理，并给出一个实际应用场景。

请输入你的回答

装饰器在Python中本质上是一个高阶函数，它接收一个函数作为参数，并返回一个新的函数（通常称为wrapper函数）。通过闭包机制捕获原函数和额外的逻辑，从而在不修改原函数源代码的前提下为其增加前置处理、后置处理、日志记录、性能统计或权限校验等附加功能。其工作原理可以拆解为三个层面：在语法层面，@decorator仅仅是func = decorator(func)的语法糖，Python解释器在定义函数时就会执行装饰器调用，将原函数名重新绑定到装饰器返回的新函数上；在实现层面，装饰器依赖于闭包——外层函数接收并保存原函数的引用，内层wrapper函数通过*args和**kwargs接收任意参数，在调用原函数前后插入增强逻辑，最后返回wrapper函数替换原函数，且内层函数必须使用functools.wraps来复制原函数的元数据（如__name__、__doc__），否则调试工具和文档生成工具会丢失关键信息；在执行时机层面，装饰器在模块导入时立即执行（而非在函数调用时），这意味着如果一个装饰器内部有耗时的初始化操作，会拖慢整个应用的启动速度，这是生产环境中需要特别注意的性能陷阱。在实际应用场景中，装饰器最常用于Web框架的权限控制，例如在Flask或FastAPI中定义一个@require_login装饰器，它会从请求头中提取JWT Token，验证用户身份并将其注入到路由函数的参数中，若验证失败则直接返回401响应，而业务逻辑函数本身只需要专注处理已认证用户的数据操作——这种做法将认证这种“横切关注点”与核心业务逻辑彻底分离，不仅让代码更加整洁，还使得权限策略的修改只需在装饰器内部调整一次，就能影响所有受保护的路由，同时配合带参数的装饰器（如@require_role('admin')）还能实现细粒度的基于角色的访问控制，这种模式在微服务架构中已经成为标准的API安全实践方案。

提交回答

评分

95

回答非常出色，展现了深厚的技术功底和工程实践经验。技术正确性上，对装饰器本质（高阶函数、闭包）、语法糖、执行时机（模块导入时）以及functools.wraps的必要性都阐述得准确且深入。完整性方面，从原理到实现再到执行时机，层层递进，逻辑严密。表达清晰，专业术语使用得当。工程实践意识尤为突出，不仅指出了性能陷阱（初始化耗时拖慢启动），还给出了Web框架权限控制这一经典且具体的场景，并延伸到了带参数装饰器和微服务架构，体现了对横切关注点分离和代码可维护性的深刻理解。唯一可提升之处是未提及类装饰器或装饰器在单元测试（如mock）中的应用，但瑕不掩瑜。

Interview Agent(b): 面试题回答与评分测试

六、系统升级与扩展

虽然 CareerPilot 已经能够完成简历分析、岗位分析、能力差距评估、学习路线规划以及模拟面试等核心功能，并形成较为完整的求职辅助闭环，但从实际使用体验来看，系统仍然存在较大的优化空间。项目当前版本更接近一个可运行的 MVP (Minimum Viable Product)，验证了多 Agent 协同解决求职问题的可行性，而未来的重点则在于提升智能化程度、用户体验以及系统扩展能力。

未来可以加入更多不同功能的 Agent。从架构角度来看，目前系统中的各个 Agent 已经实现了解耦设计，每个 Agent 都具有独立职责，并通过统一的数据结构进行通信。这种设计使得未来扩展新的 Agent 变得相对容易。例如可以增加 Resume Optimization Agent，对用户简历进行自动润色和优化；增加 Career Recommendation Agent，根据用户技能画像推荐适合的发展方向；增加 Job Crawling Agent，自动从招聘网站获取最新岗位信息，实现从岗位发现到求职准备的完整流程。由于当前系统已经采用模块化架构，因此新增 Agent 不需要大规模修改现有代码，而只需按照统一接口规范接入即可。

在 AI 能力方面，目前系统主要依赖单个大模型完成推理任务。未来可以进一步引入多 Agent 协作机制，将不同任务交给不同角色处理。例如 Resume Agent 专门负责信息抽取，Gap Analysis Agent 专门负责能力评估，而 Learning Agent 则负责规划学习路径。通过 LangGraph 等框架实现 Agent 之间的状态传递和协同决策，可以进一步提升复杂任务的处理能力。此外，还可以引入长期记忆机制，将用户历史学习记录、历次面试成绩以及职业成长轨迹存储到向量数据库中，使系统能够根据用户长期变化动态调整建议，而不仅仅基于当前一次输入进行分析。

从用户体验角度来看，学习路线规划功能仍然存在较大的改进空间。在当前版本中，学习周期完全由大模型自主决定，用户无法主动调整学习时长。这种设计虽然能够根据技能缺口自动生成学习计划，但忽略了不同用户在时间安排上的差异。例如部分用户处于暑假或求职空档期，能够投入较多时间学习，希望在更长周期内系统性提升能力；而另一部分用户可能面临即将到来的面试或秋招，需要在较短时间内集中补齐关键技能。因此未来版本可以增加学习周期配置功能，在生成学习路线之前允许用户选择“快速冲刺模式”“标准模式”或“长期提升模式”，甚至直接输入可支配学习时间，由系统结合技能缺口动态调整学习任务密度和学习节奏，从而生成更加个性化的成长路线。

前端交互体验同样值得进一步优化。目前系统已经实现完整功能流程，但界面展示仍然偏向开发原型风格。例如简历解析和岗位分析结果直接以结构化 JSON 形式展示，虽然便于开发调试，但普通用户阅读时体验较差。未来可以将技能、项目经历和教育背景设计为卡片式展示，将能力匹配度结果设计为可视化仪表盘，并通过柱状图、雷达图等方式展示用户与岗位之间的能力差距。同时学习路线可以采用时间轴形式呈现，让用户更加直观地理解学习进度和阶段目标。对于面试模块，还可以增加聊天式交互界面，使用户能够获得更加接近真实面试场景的体验。

此外，目前系统主要采用用户手动输入职位描述的方式进行分析。在后续版本中，可以增加岗位链接解析功能，用户只需输入招聘网站链接即可自动抓取职位信息并完成分析。进一步还可以接入招聘平台 API，实现实时岗位推荐和自动匹配功能，使系统从“求职分析工具”逐步演化为“智能求职平台”。

从工程化角度来看，目前项目已经具备 Docker 部署能力和基础测试体系，但在可观测性和运维方面仍有提升空间。未来可以接入 LangSmith、OpenTelemetry 等工具，对 Agent 调用链路、大模型响应时间以及 Prompt 效果进行追踪和分析。同时建立更加完善的 Benchmark 数据集，对不同版本 Prompt、不同模型以及不同 Agent 设计方案进行量化评

估，从而实现持续优化和迭代。

总体而言，CareerPilot 当前版本已经验证了 AI Agent 在求职场景中的应用价值，但仍有广阔的发展空间。未来的发展方向不仅仅是增加功能，更重要的是让系统具备更强的个性化能力、更自然的人机交互体验以及更完善的长期职业成长支持能力，使其真正成为能够陪伴用户持续成长的智能职业助手。

七、课程总结

回顾整个课程和项目开发过程，我最大的感受是自己对 AI 开发的认知发生了非常大的变化。在课程开始之前，我对于 AI Agent、LangGraph、Function Calling、MCP 等概念几乎没有了解，也没有实际接触过相关开发工作。虽然平时会使用 ChatGPT、GitHub Copilot 等工具辅助学习和编程，但更多只是把它们当作代码补全工具，而没有真正理解 AI 在软件工程中的作用。最初确定项目选题时，我甚至不清楚一个 Agent 系统究竟应该如何设计，也不知道应该如何组织项目结构、如何调用大模型接口以及如何完成测试和部署。

真正开始开发之后，我逐渐意识到 AI 开发与传统的软件开发既有相似之处，也有明显不同。过去学习编程时，我更关注具体代码如何实现，而在本次项目中，我发现决定项目质量的往往不是某一段代码，而是需求是否清晰、架构是否合理以及模块之间的协作方式是否明确。例如在项目初期，我按照课程要求先编写了 requirements.md、architecture.md 和 tasks.md 等规格文档，然后再利用 AI 辅助生成代码。这个过程让我第一次切身体会到规格驱动开发（SDD）的价值。当需求和架构被提前定义清楚之后，AI 生成代码的质量明显更高，后续修改成本也更低。相比直接让 AI 从一句简单描述开始生成整个项目，先完成规格设计实际上能够大幅提高开发效率。

在工程能力方面，本次项目也让我接触到了很多过去很少关注的问题。项目开发过程中并不是一次生成代码就能够成功运行，而是经历了大量调试和排错过程。例如缺少依赖导致模块无法导入、SQLite 数据表初始化失败、pytest 测试环境配置问题、DeepSeek 接口兼容性问题、前后端状态同步问题以及大模型返回格式不符合预期等。这些问题往往与算法本身无关，而是典型的软件工程问题。通过逐步定位问题、分析日志、修改代码并重新验证，我对软件开发完整生命周期有了更加深刻的理解，也认识到一个能够稳定运行的系统远比单纯生成代码更加重要。

与此同时，我对 AI 编程工具的认识也发生了改变。课程开始时，我认为 AI 可以直接替代大部分开发工作，只需要提供需求描述就能够自动生成完整项目。但在实际开发过程中发现，AI 更像是一位能力很强但缺乏上下文的协作者。如果需求描述不清晰，或者开发者无法判断生成结果是否正确，那么 AI 生成的代码同样可能存在问题。项目中出现的测试失败、数据库异常以及接口调用错误都说明，人类开发者仍然需要负责需求分析、架构设计、结果验证和问题排查。AI 的价值在于提高开发效率，而不是完全替代工程师。因此，未来软件工程师的重要能力可能不再只是编写代码，而是能够清晰表达需求、设计系统架构并有效协同 AI 完成开发任务。

对于课程本身，我整体上非常认可。本课程不仅介绍了 AI Agent、大模型应用等前沿技术，更重要的是通过完整项目实践，让学生真正体验了从需求分析、架构设计到开发测试和部署的全过程。相比单纯学习理论知识，这种以项目驱动的教学方式让我对 AI 原生开发和软件工程实践有了更加深刻的理解。在完成 CareerPilot 项目的过程中，我不仅掌握了 Agent 系统开发的基本流程，也体会到了规格驱动开发和 AI 协同编程的价值。未来如果课程能够持续结合 AI 领域最新的发展趋势，引入更多真实应用案例和行业实践内容，将能够帮助学生进一步了解 AI Agent 技术在不同场景中的应用潜力。总体而言，这门课程让我收获颇丰，也让我对未来 AI 软件开发的发展方向有了更加清晰的认识。

总体而言，本次课程项目不仅让我完成了一个能够实际运行的 AI Agent 系统，更重要的是让我建立了面向 AI 时代的软件工程思维。从最初对 Agent 技术几乎一无所知，到最终完成 CareerPilot 的设计、开发、测试和部署，我深刻体会到 AI 正在改变软件开发方式，而工程化能力、需求分析能力以及与 AI 协作的能力将变得越来越重要。这些收获不仅帮助我完成了本次课程项目，也为未来继续学习和应用 AI 技术打下了良好的基础。